End-to-End AI-Based System for Vehicle License Plate Recognition

Fedi Sannoura, Hamadi Chihaoui, Fethi Filali
Qatar Mobility Innovations Center (QMIC)
Qatar University
Doha, Qatar
sonnara.fedi@gmail.com, chihaoui.hamadi@gmail.com, filali@qmic.com

***Abstract*—The design and implementation of an end-to-end automatic license plate recognition (ALPR) system are presented in this paper. ALPR can be conceptualized as a series of two sequential object detection tasks; consequently, a preliminary overview of this research area was conducted, including problem formulation, detector families, and performance metrics. Subsequently, a comprehensive investigation of the state-of-the-art in ALPR was conducted, examining the various components of ALPR systems as well as a variety of techniques and data sets. Particular attention was paid to end-to-end methodologies based on deep learning, as our proposed contribution falls within this context. In the end, a deep learning-driven end-to-end model was developed, trained, and optimized to meet the requirements of the intended ALPR system. The resulting model exhibits near-real-time execution speeds, high accuracy, and robust generalization capabilities, demonstrating its practical applications potential.**

***Index Terms*—Automatic License Plate Recognition (ALPR), Object Detection, Deep Learning, End-to-End Model**

I. Introduction

AUTOMATIC vehicle license plate detection and recognition play a crucial role in the modernization of cities and intelligent transportation systems. Boasting a wide range of applications, from access control and law enforcement to human well-being, this field has attracted significant research efforts in recent years. The advent of deep learning and its application to object detection has led to a surge in research contributions and the industrialization of ALPR for real-world scenarios.

Automatic license plate detection and recognition involves extracting vehicle license plate characters from a video or an image [7]. A single image may contain one or multiple license plates simultaneously. While many existing algorithms perform well under controlled conditions or with sophisticated image capture systems, accurately reading license plates in uncontrolled environments—such as those with complex backgrounds, varying photographic conditions, illumination, distortion, occlusion, or blurring—remains a challenging task.

As illustrated in Figure 1, this research topic reached its peak in terms of research output in 2011, only to decline as traditional object detection algorithms stagnated, as discussed in the previous chapter. However, with the introduction of Convolutional Neural Networks (CNNs) for object detection, ALPR experienced a resurgence in research output from 2016 to the present day. Although ALPR can be viewed as an object detection problem, it possesses unique features and constraints that grant it a certain level of independence as a research topic.

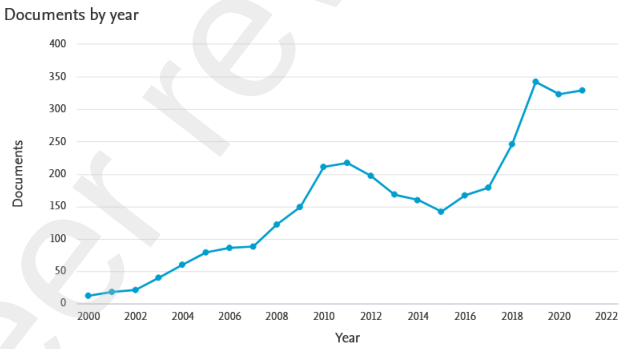

Fig. 1. Research documents by year in the field of ALPR [8]

As the primary objective of an ALPR system is to automatically extract License Plate (LP) information without requiring human intervention, this information enables the identification of both the vehicle and its owner. With these identities in hand, numerous applications can be automated, such as:

- Vehicle access control: Private parking lots, controlled zones, and limited traffic areas.
- Payment services: Toll collection, electric vehicle charging/fuel stations, and ticketless parking systems.
- Security and law enforcement: Red light enforcement, speed limit enforcement, and both mobile and stationary police surveillance.
- Community well-being: Data collection for smart city initiatives.

Despite the remarkable progress observed in ALPR, numerous challenges remain for real-world applications, particularly under uncontrolled conditions, as depicted in Figure 2. These challenges include:

- Accuracy: Ensuring high levels of recognition accuracy is crucial for the effectiveness of an ALPR system. A system with low accuracy may lead to misidentification of vehicles, which could have serious consequences in applications such as law enforcement or access control.
- Model size and execution speed, particularly for edge devices and smart camera applications.
- Tilt angle of the license plate in the captured image.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

- Image quality and distortions.
- Environmental factors, such as illumination (day/night) and weather conditions (sunny/cloudy).
- License plate properties, including texture, background/character color, font size, plate size, and the number/type of characters.

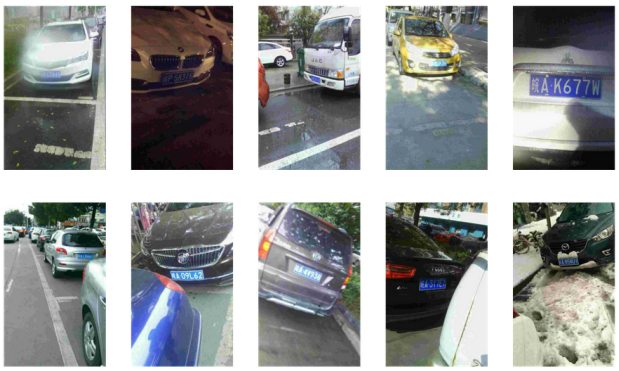

Fig. 2. Challenging ALPR images (source: CCPD dataset [13])

To recognize a license plate, two primary tasks must be executed: license plate detection and license plate character recognition.

License plate detection is the initial ALPR task, aiming to localize the license plates within an image by identifying bounding boxes or corner points, as depicted in Figure 3. Existing methods can be broadly divided into three categories: traditional techniques (pre-CNN era), which employ various computer vision techniques such as edge detection, morphological operations, and pattern matching; CNN-based license plate detection, which utilizes Convolutional Neural Networks to identify and localize license plates more effectively; and end-to-end CNN-based license plate detection, which leverages a single network to simultaneously address both license plate detection and character recognition tasks, often yielding improved performance and efficiency.

License Plate Character Recognition is the second ALPR task. Provided with the license plate location within an image, its objective is to identify the displayed characters. Existing methods can be categorized into the same three groups mentioned in license plate detection: traditional techniques (pre-CNN era), CNN-based methods, and end-to-end CNN-based approaches. It is also worth noting that license plate recognition techniques can be further classified into Segmentation-Based and Segmentation-Free methods.

Fig. 3. License plate bounding box (red) vs corner points (green) [13]

Segmentation-Based Methods are approaches that begin by segmenting image pixels, as demonstrated in 4, before proceeding to character recognition. Character segmentation relies on the fact that characters and backgrounds exhibit distinct colors on a license plate. Typically, a binary image of a license plate is generated to obtain character boundaries through horizontal pixel projection. The problem is then transformed into single character recognition, which is a form of image classification task. A major drawback of such methods is their heavy dependence on character segmentation quality, which is sensitive to unconstrained conditions. As a result, even with a robust recognizer, improper segmentation can hinder accurate plate recognition..

Fig. 4. Example of license plate segmentation [3]

Segmentation-Free Methods directly proceed to character recognition without explicit segmentation, an approach widely adopted with the evolution of deep neural networks.

Although ALPR systems have achieved impressive results, most existing contributions focus on addressing license plate location and character recognition as separate sub-tasks. This strategy increases model size and reduces execution speed due to redundancy while accumulating errors from independent sub-problems, leading to lower overall accuracy.

To overcome these limitations, end-to-end deep learning-based ALPR was proposed by [5] and further enhanced by [13] and [9]. However, these contributions either lacked generalization or were slow when considering edge device deployment. In this work, the goal was to design and implement a near real-time end-to-end ALPR system, building upon previous research in the same context. Subsequently, the designed model was trained and optimized to accelerate execution time, targeting the Jetson Nano edge device.

The remainder of this paper is structured as follows: Section II discusses related works, highlighting the key differences between various ALPR techniques. Section III delves into the model design, detailing the architecture and the components employed. Section IV focuses on the model's training and optimization processes to ensure efficient performance. Section V presents the results achieved by the proposed model, comparing its performance to existing approaches. Finally, Section VI offers concluding remarks and outlines potential avenues for future work in the ALPR domain.

II. Related Work

A. Traditional ALPR Techniques

In the literature, traditional ALPR refers to techniques developed before the introduction of CNNs. These systems are multi-stage, implying that ALPR sub-tasks are addressed

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

separately. Moreover, they rely on hand-crafted feature extraction and exhibit limited generalization to unconstrained scenarios. In the following paragraphs, we will present the most influential traditional license plate detection and recognition techniques.

1) License Plate Detection

Traditional license plate detection techniques rely on hand-crafted features and shallow trainable architectures. These methods exploit the fact that license plates have rectangular shapes with special colors and relatively length-fixed characters. Therefore, edge-based, color-based, texture-based, and character-based methods have been adopted.

One example of an edge-based method is the edge clustering approach presented in [20], which uses Expectation Maximization (EM) to extract the regions of license plates. Another edge-based method is the line density filter approach proposed in [33], which extracts candidate regions using linear Support Vector Machine (SVM) based on color salience features.

Color-based methods use color pixels to detect the location of a license plate based on a modified template-matching technique, as proposed in [17]. These methods are usually fast enough for real-time applications. However, they are sensitive to illumination conditions in unconstrained scenes. To overcome this problem, a robust texture-based method is proposed in [30] using wavelet transform and empirical mode decomposition (EMD) analysis to search for the location of a license plate.

Multi-level extended local binary pattern is used as an efficient detector in [26]. This type of method can alleviate the problems caused by illumination changes and complex background, which is achieved by using more time-consuming processing modules compared to edge-based and color-based methods.

Character-based methods, as presented in [29] and [15], directly locate the alphanumeric characters of the car plate using character features. In [15], the license plate is decomposed into several constituent characters, and conditional random field (CRF) and maximally stable extremal region (MSER) are used to extract and construct characters, respectively. These methods can adapt to various environmental factors to some extent, but they can be easily affected by character-like background.

In summary, traditional license plate detection techniques use a variety of methods to detect license plates, including edge-based, color-based, texture-based, and character-based methods. Each method has its advantages and disadvantages, and the choice of method depends on the specific application requirements.

2) License Plate Recognition

Before the widespread use of deep learning technology (DL), traditional methods for license plate recognition were typically based on character segmentation, followed by optical character recognition (OCR) methods for character classification. In [33], traditional methods were reviewed and compared from different perspectives.

Template matching and learning-based methods are two commonly used methods for character recognition. In [19], segmented images are compared with predefined standard templates, and characters are recognized based on the best match. In [16], a hybrid discriminative restricted Boltzmann machine is trained for character classification. Traditional machine learning methods, such as HMM (hidden Markov Models) and SVM (Support Vector Machine), can also be used for classification.

Segmentation-free methods typically extract license plate (LP) character features directly to avoid segmentation or deliver the LP to an OCR system to perform the recognition task. In contrast, segmentation-based methods require determining the LP bounding box and applying shape correction before segmentation. Various features of LP characters can be utilized for segmentation, such as Connected Components Analysis (CCA) and character-specific extremal regions. After segmentation, the LP is fed to a hand-crafted feature extractor, such as SIFT, and then to a classifier, such as SVM, as a final step for character recognition.

Deep learning-based methods have become increasingly popular for license plate recognition, especially with the advent of convolutional neural networks (CNNs). These methods can learn feature representations directly from the raw image data, avoiding the need for manual feature engineering. Moreover, end-to-end approaches can directly predict the LP characters without explicit character segmentation, simplifying the recognition process. However, these methods typically require large amounts of annotated training data and powerful hardware for training and inference, which can be computationally intensive.

B. Deep Learning Based Multi-Stage ALPR Techniques

With the introduction of convolutional neural networks (CNNs) in computer vision, many models were proposed as an enhancement to traditional techniques thanks to their outstanding performance and strong generalization capabilities in uncontrolled conditions. Deep-learning (DL) based license plate (LP) detection and recognition have emerged as two separate sub-tasks.

DL-based LP detection methods can be divided into two categories: two-stage methods and one-stage methods. Two-stage methods first generate candidate LP regions using a region proposal network (RPN) and then classify the candidate regions as LP or non-LP using a classifier. One-stage methods directly regress the LP region from the input image. Both approaches have achieved state-of-the-art performance in LP detection, with one-stage methods being faster and more efficient.

DL-based LP recognition methods can be further divided into segmentation-based and segmentation-free methods. Segmentation-based methods require segmenting individual characters in the LP, followed by character recognition using CNNs. Segmentation-free methods directly recognize the LP characters without explicit character segmentation, typically

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

using CNNs and sequence modeling methods, such as recurrent neural networks (RNNs) or attention mechanisms. End-to-end approaches, which directly predict the LP characters from the input image, have become popular in recent years and have shown strong performance.

DL-based methods have shown remarkable performance improvements over traditional techniques, especially in uncontrolled conditions, such as variable lighting, motion blur, and occlusions. However, these methods require large amounts of annotated training data and powerful hardware for training and inference, which can be computationally intensive. Furthermore, DL-based methods may not be as interpretable as traditional techniques, which could be important for certain applications

1) License Plate Detection

In recent years, object detection methods based on deep neural networks have become widely adopted, benefiting from the development of deep learning technology and computing power. DL-based methods typically extract features and regress location parameters with deep neural networks, mostly built with supervised training.

For feature extraction, most DL-based methods adopt commonly used backbones, such as DarkNet, ResNet (50, 101, 152), VGG, and DenseNet, which can be time-consuming. Some methods design lightweight networks for efficiency, such as the network proposed in [28], which has less than 20 convolutional layers and is fast enough for real-time applications.

For location, generic object detection methods, such as YOLO [6], SSD [12], and Faster-rcnn [11], can be used to obtain the bounding box of a license plate. Anchor-based one-stage methods, such as YOLO and SSD, are frequently used for license plate detection due to their lower computation cost. For example, [27] presents a robust and efficient ALPR system using YOLO detector, and in [25], a multi-directional detector based on the modified YOLO framework is proposed for license plate images.

Compared with anchor-based methods, anchor-free methods [24] avoid hyper-parameters related to the anchors and calculation of Intersection over Union (IoU) between ground truth boxes and anchor boxes, which makes the training process more efficient and easier to train. In addition, some tricks can improve performance. For example, a hybrid cascade structure is designed for detecting small and vague license plates in complex scenes in [23], and RNN is designed to improve the locating rate of license plates in complex scenes in [22]. In summary, deep learning-based methods are often more robust by training with a large number of samples.

2) License Plate Recognition

Benefiting from the development of deep learning technology and computing power in recent years, segmentation-free methods have gradually become the prevailing methods for license plate recognition. These methods avoid the uncertainty of character segmentation, and usually take the license plate characters as a sequence directly, transforming the recognition problem into a sequence labeling problem.

To solve this type of problem, both CNN-based and Recurrent Neural Network (RNN) based methods have been adopted in different situations. In [34], a CNN classifier is employed for character recognition directly. In [32], a CNN learns deep features, and then a kernel-based ELM (Extreme learning machine) classifier is utilized for classification. Other methods, such as Fully Convolutional Network (FCN) based methods [31], which find each character area by semantic segmentation, can also achieve relatively good results.

RNN-based methods are widely used in speech recognition, language modeling, machine translation, etc. RNNs have natural advantages for sequence labeling due to their powerful connectionist mechanism. In [14], a bi-directional recurrent neural network (BRNN) is adopted for solving sequence labeling of license plate characters, and the recognition is accomplished by the BRNN and connectionist temporal classification (CTC) [18]. In [21], a Long Short-Term Memory (LSTM) network is trained to recognize the extracted sequential features from the license plate via CNNs. In [21], an RNN is designed to automatically recognize the characters of Chinese license plates. However, RNN-based methods depend on the computations of the previous time step and therefore do not allow parallelization over every element in a sequence.

In summary, segmentation-free methods for license plate recognition have become a hot research topic in recent years, and both CNN-based and RNN-based methods have been adopted in different situations. These methods offer the advantage of avoiding character segmentation uncertainty, but may require a large amount of training data and computational resources. RNN-based methods have natural advantages for sequence labeling due to their powerful connectionist mechanism, but may not allow parallelization over every element in a sequence.

C. End-to-End ALPR Techniques

In contrast to the previously mentioned approaches that consider license plate detection and recognition as two independent tasks and solve them sequentially, end-to-end Automatic License Plate Recognition (ALPR) jointly solves both tasks by a single deep neural network based on the fact that LP detection and recognition are highly correlated. This approach not only accelerates processing speed but also avoids intermediate error accumulation.

Accurate bounding boxes obtained via detection methods can improve recognition accuracy, while the recognition result can be used to eliminate false positives vice versa. Moreover, operations between different stages, such as extracting and resizing the LP region for recognition, are always accomplished by less efficient CPU, making LP recognition slower.

Last but not least, thanks to feature sharing between detection and recognition, end-to-end models require fewer parameters compared to using separated models, resulting in a gain in model size and execution time. To the best of our knowledge, [5], [9], [13] are the unique contributions in this context.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

In the following paragraphs, each of these contributions will be presented in detail.

1) TE2E - Toward End to End

The first work that integrates both license plate detection and recognition into a single network and solves them at the same time was presented in [5]. This approach can be trained end-to-end, with only the image, plate positions, and labels needed for training. The resulting system achieves high accuracy on both plate detection and letter recognition.

The whole framework takes 0.3-0.4 seconds per image on an NVIDIA Titan X GPU with 12GB memory, which is relatively fast compared to traditional methods that use separate models for detection and recognition. This end-to-end approach avoids intermediate error accumulation and takes advantage of feature sharing between detection and recognition. Overall, [5] provides a strong foundation for further development of end-to-end ALPR systems.

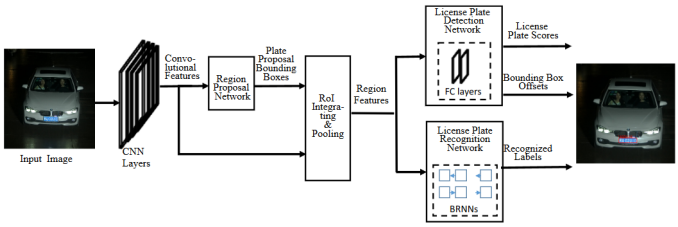

Fig. 5. Model Proposed by [5]

The proposed model, as shown in Figure 5, is similar to Faster-RCNN and starts with VGG-16 as the feature extractor and region proposal stage that outputs 256 proposals. The model then creates a bounding box that integrates all proposals that refer to the same object.

Next, it max-pools the feature maps of each proposed bounding box to a 4x20 grid. Finally, the model provides three outputs: the confidence of the license plate, the offset on the bounding box after regression, and characters using a bidirectional recurrent neural network (BRNN) with connectionist temporal classification (CTC) technique.

Overall, this model integrates detection and recognition into a single network and achieves high accuracy on both tasks. The use of a BRNN with CTC technique for character recognition allows for efficient and accurate sequence labeling.

2) RPNet - RoadSide Parking Network

The second work in this context was presented in [13] and was named the Roadside Parking net (RPnet). This model was motivated by the fact that the previous work achieved a low Frames Per Second (FPS) despite the high computational performance of the Titan X GPU. RPnet achieved more than 60 FPS on a weaker hardware (NVIDIA Quadro P4000) without loss of accuracy.

In addition, the authors introduced CCPD, the largest and most diverse publicly available dataset for ALPR to date. CCPD provides more than 250k unique car images with detailed annotations, which greatly benefits the training and testing of ALPR models. Overall, RPnet and the CCPD dataset provide important contributions to the development of efficient and accurate end-to-end ALPR systems.

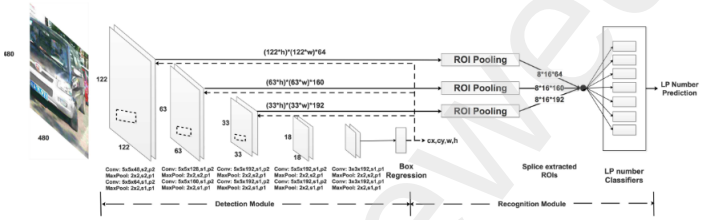

Fig. 6. Model Proposed by [13]

The model proposed in [9] starts with a Single Shot Detector (SSD) network to extract LP bounding boxes, as shown in Figure 6. After that, the model pools the feature maps associated with detected bounding boxes to an 8x16x416 map. It's worth noting that the feature maps are extracted from the high convolutional layers (layer 1+3+5), as experiments showed that these layers provide the best recognition results.

Finally, a fully connected layer is used to recognize each character in the license plate. The authors demonstrated that their proposed model outperforms other approaches in both accuracy and speed, and that higher layers matter in the recognition module (multi-layer feature maps for recognition). They also found that feature extraction from more layers does not necessarily increase accuracy. Overall, the proposed model in [9] provides a strong example of an efficient and accurate end-to-end ALPR system.

3) Anchor-Free Net

Although end-to-end ALPR models have been proposed in the literature before, [9] was the first end-to-end anchor-free model, as shown in Figure 7.

For feature extraction, the authors used ResNet-18 as a lightweight backbone and a feature pyramid network to obtain the feature map. After that, a detection head extracts both bounding boxes and license plate corners. This information is used to rectify the LP shooting angle. Finally, characters are recognized using a CNN+CTC technique based on the previously extracted feature map.

The proposed model in [9] provides an efficient and accurate solution for end-to-end ALPR, and the use of an anchor-free approach improves computational efficiency while maintaining accuracy. This work represents an important contribution to the development of more efficient and accurate ALPR systems.

III. Model Desing

A. Target ALPR System Requirements

The requirements for ALPR models can vary depending on the target application, with edge devices having less storage and computation power than cloud environments, which can affect the achievable speed and accuracy. In our context, the following minimum requirements are necessary for our ALPR model:

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

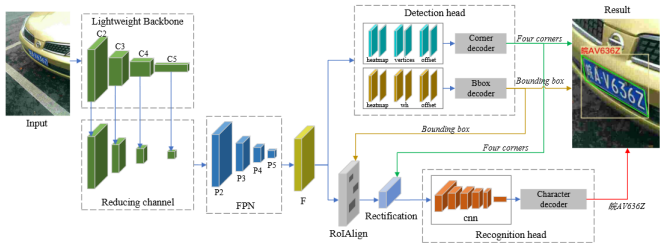

Fig. 7. Model Proposed by [9]

a) Architecture:

The model must be end-to-end, with a shared feature map between the detection and recognition tasks, reducing model size and computation requirements. Using a single loss function that includes both detection and recognition losses eliminates error accumulation between tasks.

b) Generalization:

The model must not make assumptions about LP properties such as color or font, or shooting conditions such as distance, angle, or illumination, to ensure robustness in different scenarios.

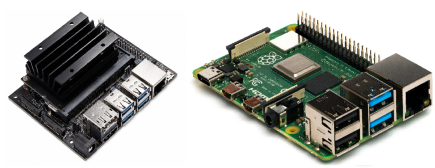

Fig. 8. Examples of edge devices (source: Nvidia and Raspberry Pi websites)

c) Target device:

The model must run on lightweight edge devices like Raspberry Pi and Jetson Nano, which can process real-time captured data. Running the model directly on the edge device avoids communication link overload, security, and scalability issues associated with transferring data to the cloud.

d) Speed:

The model must achieve near real-time speed of at least 10 FPS to ensure captured vehicles are not missed between frames. This enables post-processing techniques like LP tracking and voting mechanisms to enhance accuracy.

e) Accuracy:

The model must achieve at least 90% accuracy, which can be improved through post-processing techniques such as LP tracking and voting mechanisms.

B. Proposed Model

1) Model Architecture

After conducting a thorough literature review, the following points must be given significant consideration during the model design process, as they summarize the conclusions drawn from previous works:

- Feature Pyramid Networks (FPNs) can be used to fuse different levels of feature maps from top to bottom.
- A 1×1 convolution layer is very useful as a channel reducing strategy.
- Sharing the feature map between different tasks can increase the overall model speed without necessarily impacting the overall accuracy.
- Feature maps from relatively lower layers also matter for recognizing LP characters, as the area of the LP is expected to be very small relative to the entire image.
- Feature extraction from more layers does not necessarily increase accuracy.

For the above mentioned reasons, the proposed model is shown in Figure 9. Given an input image, a shared feature extractor is used to provide the rest of the network with the necessary feature maps. After that, two paths are employed: LP bounding box regression and LP recognition.

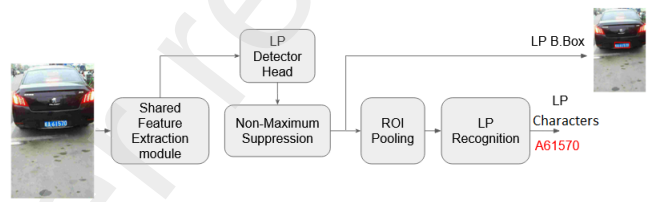

Fig. 9. Proposed model architecture

In the next subsections, each of these modules will be detailed.

2) Shared Feature Extractor

The proposed feature extractor shown in Figure 10 was inspired by [9] and adopted in our work for several reasons. Firstly, it uses a lightweight backbone, namely ResNet-18, for better speed and efficiency. Secondly, it applies channel reduction using 1x1 convolutions, which reduces the computational cost. Thirdly, it uses feature pyramid networks to mix the different feature maps and get a unique feature map that will be shared between the rest of the modules. The output of this module is $B \times \frac{1}{4}H \times \frac{1}{4}W \times 128$, where W and H represent the input image width and height, respectively, B stands for the batch size, and 128 is the depth of the feature map.

Inside each convolution block output, a 1x1 convolution kernel with 128 filters is applied for channel reduction. As a result, four layers are obtained: $H/4 \times W/4 \times 128$, $H/8 \times W/8 \times 128$, $H/16 \times W/16 \times 128$, and $H/32 \times W/32 \times 128$. In order to compensate for the difference in dimensions, the last three layers are interpolated by 2, 4, and 8 correspondingly. Then, all four layers are concatenated to obtain a $H/4 \times W/4 \times 512$ feature map. Finally, a 3x3 convolution kernel with 128 filters is applied for further channel reduction, followed by Batch Normalization (to prevent over-fitting) and ReLU activation (for speed). This yields the shared feature map of $H/4 \times W/4 \times 128$, which will be used later for both plate detection and recognition simultaneously.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

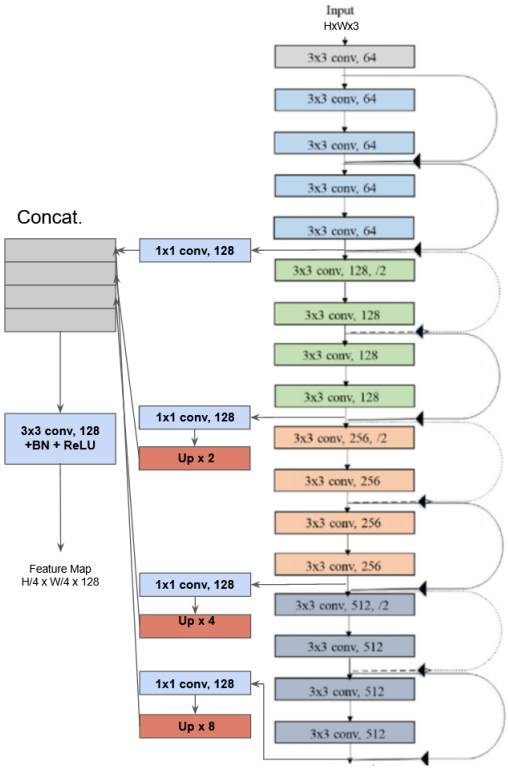

Fig. 10. Feature extractor architecture

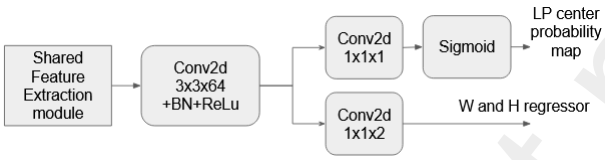

Fig. 11. LP detector

3) LP Detector

The LP bounding box regressor provides a 3D matrix, where each (x,y) coordinate returns three pieces of information: the probability of having an LP in that specific feature coordinate, as well as the estimated width and height of the bounding box. The LP detector is illustrated in detail in Figure 11. It is anchor-free, so as to avoid the need to tune bounding box hyper-parameters. In the figure, BN stands for Batch Normalization, and ReLU stands for the rectified linear unit activation function. Batch Normalization is used to prevent model over-fitting, while ReLU is used for efficiency, as it introduces non-linearity to the convolution layer without incurring heavy computational costs.

4) Non-Maximum Suppression

After obtaining the LP bounding box regressor output matrix, non-maximum suppression is applied to filter redundant results. In other words, boxes whose Intersection over Union (IoU) with the box having the highest confidence is greater than 0.5 are eliminated. This is done to avoid selecting overlapping boxes that could refer to the same LP. The peak of each region is then used as the final license plate bounding box center. Finally, the width and height of the bounding box are extracted from the corresponding coordinates in the W and H regressors that were obtained from the LP bounding box regressor, as presented in figure 11.

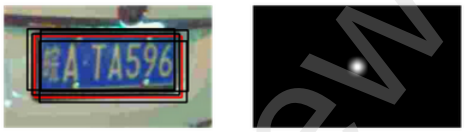

Fig. 12. Non Maximum suppression input - right: detected LP neighbourhood, left: Max probability (red) and randomly chosen positives (black)

5) Region of Interest Pooling

Region of Interest (RoI) pooling is an important step in license plate recognition. It involves creating a fixed-size representation of the LP bounding box proposal. The operation is done by splitting the input feature map into a grid with a target size and then extracting a single number from each grid element, such as the maximum or average. This ensures a fixed-size output regardless of the input size. Another technique in the same context is RoI Align, which provides better results in terms of feature maps boundary accuracy, but is heavier and slower to execute compared to RoI Pooling.

In the proposed model, RoI Pooling is used to create a fixed-size representation of the LP bounding box proposal from the original feature map associated with it. The output of RoI Pooling is a new feature map of size $32 \times 96 \times 128$ (height x width x depth). The original feature maps are max-pooled per each grid element in this case.

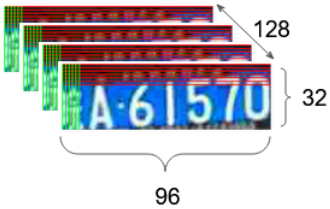

Fig. 13. Grid applied on feature map in RoI pooling

6) Characters Recognition

In the character recognition phase, the model identifies the characters on the license plate. In the proposed model, this task is approached as a segmentation-free sequence labeling problem. This approach is proven to provide significant performance and more robustness to changes. As shown in Figure 14, the character recognizer takes a feature map of size $32 \times 96 \times 128$ as input and outputs a matrix of size $24 \times K$, where 24 is the number of time steps and K is the number of character categories plus one (the empty character). In this work, K is set to 38 as the network is trained on the CCPD dataset. For example, in the case of Tunisian and Qatari license

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

plates, K should be set to 11 as these plates only contain numbers. To solve the labeling problem, the beam search algorithm is adopted for a better trade-off between search time and accuracy.

```
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_50 (Conv2D)          (None, 32, 96, 128)       147584

 max_pooling2d_20 (MaxPoolin  (None, 32, 48, 128)      0
 g2D)

 conv2d_51 (Conv2D)          (None, 32, 48, 128)       147584

 max_pooling2d_21 (MaxPoolin  (None, 32, 24, 128)      0
 g2D)

 conv2d_52 (Conv2D)          (None, 32, 24, 128)       147584

 conv2d_53 (Conv2D)          (None, 32, 24, 128)       147584

 conv2d_54 (Conv2D)          (None, 32, 24, 38)        4902

 conv2d_55 (Conv2D)          (None, 1, 24, 38)         46246

=================================================================
Total params: 641,484
Trainable params: 641,484
Non-trainable params: 0
_________________________________________________________________
```

Fig. 14. Character recognition head

IV. Training and Optimization

A. Model Training Strategy

1) Loss Function

The loss function of a multi-task model is a combination of multiple terms, as shown in equation 1:

$$L = (1-\alpha) \times L_d + \alpha \times L_r \tag{1}$$

Here, L_d refers to the LP detection loss, L_r refers to the recognition loss, and α is a parameter that controls the loss function focus, which will be detailed in the training section.

The detection loss (L_d) is the sum of the losses on the centre probability and width and height, as shown in equation 2, where L_{centre} is the sum of the logistic losses, and L_{width} and L_{height} are the smooth L1 loss [10] between the width and height ground truths and regressions.

$$L_d = L_{centre} + L_{width} + L_{height} \tag{2}$$

The recognition loss (L_r) is calculated using the CTC loss function [1], as shown in equation 3. Here, S is defined as the training dataset, x is the input, z is the target sequence, $P(z|x)$ is the conditional probability, L (set to 24) is the length of the decoding sequence, and $p^t_{\pi_t}$ is the probability of a sequence π_t at time t.

$$L_r = -\sum_{(x,z)\in S} \ln P(z|x) = -\sum_{(x,z)\in S} \ln \prod_{t=1}^{L} p^t_{\pi_t} \tag{3}$$

[1]CTC loss is explained in detail in [1]

The ground truth is encoded in the file names and parsed when loading the requested training batch. The resulting matrices include the license plate location heatmap, which consists of inserting the pattern shown in 15 (where its center matches the LP center) in a zero-initialized array. This creates a black image with a white blurred dot in the LP center, as shown in 12. This approach provides the model with more flexibility by providing it with multiple positives instead of a single one (the exact LP center) and trains the model to give confidence values around the LP center, which aids non-maximum suppression.

```
[[  0   0   0   0   0   0   0]
 [  0   0  16  42  16   0   0]
 [  0  16 114 187 114  16   0]
 [  0  42 187 255 187  42   0]
 [  0  16 114 187 114  16   0]
 [  0   0  16  42  16   0   0]
 [  0   0   0   0   0   0   0]] /255
```

Fig. 15. License plate detection heat-map

In order to train the license plate detection module, the ground truth values for the width and height of the license plate are also needed. These ground truth values are extracted from the file name in the same way as the LP centre location. A pattern, as shown in Figure 16, is then inserted in both the width and height ground truth matrices. The centre of this pattern is made to match the LP centre, similar to the LP centre location heat-map. The purpose behind this step is to give the model multiple positive samples instead of a single one, which helps the model to learn to detect the license plate with variations in the width and height.

```
[[  0   0   0   0   0   0   0]
 [  0 262 262 262 262 262   0]
 [  0 262 262 262 262 262   0]
 [  0 262 262 262 262 262   0]
 [  0 262 262 262 262 262   0]
 [  0 262 262 262 262 262   0]
 [  0   0   0   0   0   0   0]] /720
```

```
[[  0   0   0   0   0   0   0]
 [  0 120 120 120 120 120   0]
 [  0 120 120 120 120 120   0]
 [  0 120 120 120 120 120   0]
 [  0 120 120 120 120 120   0]
 [  0 120 120 120 120 120   0]
 [  0   0   0   0   0   0   0]] /1160
```

Fig. 16. Pattern for width (left) and height (right) ground truth matrices

2) Training Process

The proposed model was trained using the Adam optimizer for 38 epochs with mini-batches of 32 images. The stopping criteria for the training process was the plateau on the validation loss for three successive epochs. The parameter α was set to 0.9, as in [9], to give the recognition loss the highest weight. The initial learning rate was set to 10^{-3} and was divided by 10 every 10 epochs. Due to limited resources, the model was trained on a subset of 30k images from the CCPD dataset, with an additional 2k images reserved for validation. These 32k images were kept constant throughout the entire training process.

During each epoch, the resulting model was saved with the epoch's number appended to its name. The training loss dropped significantly after epoch 15, as shown in Figure 17. At this stage, the model started to converge towards its optimum. Correspondingly, the training and validation accuracy began to

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

increase significantly, as shown in Figure 18. The performance of the model on the validation set was used to monitor overfitting and decide when to stop the training process.

Overall, the training process allowed the model to learn to accurately detect and recognize license plates in images, as will be demonstrated in the evaluation section.

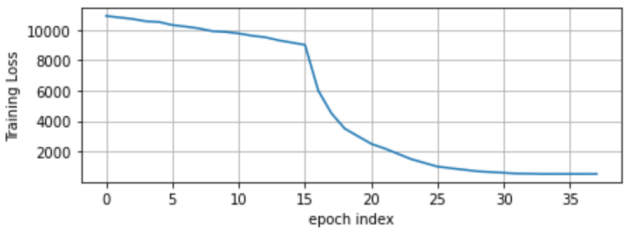

Fig. 17. Training loss vs epoch index

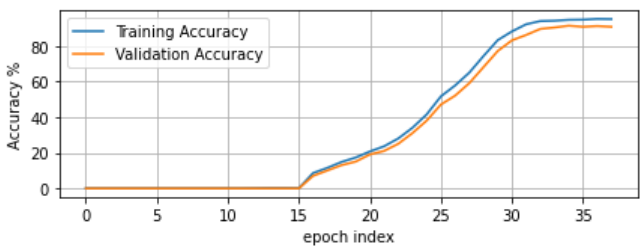

Fig. 18. Training and validation accuracy vs epoch index

At epoch 35, we observe that the training loss no longer significantly decreases, and both the training and validation accuracy reach a plateau. This indicates that the detection network is generating useful proposals, and the recognition network is learning to accurately label the characters based on the detection result. It is worth noting that during the training process, the model has to balance between detection and recognition tasks, which can be challenging due to their different nature and requirements.

To avoid overfitting and ensure good generalization, the training was stopped after three successive epochs with no improvement on the validation loss. The resulting model was then evaluated on the test dataset, achieving an accuracy of 90.6%. This indicates that the proposed approach achieves high accuracy and robustness in recognizing license plates from diverse regions and with various fonts and designs. It also demonstrates the effectiveness of the proposed multi-task learning strategy, where the detection and recognition tasks are jointly optimized, leading to better results compared to training them independently.

B. Development Environment

The development environment utilized during in this work included the following tools and technologies:

- Python: Python is a popular high-level, interpreted, general-purpose programming language. It is widely used in the field of machine learning and artificial intelligence due to its dynamic-typing and multiple programming paradigms support, including structured (particularly procedural), object-oriented and functional programming. Its flexibility and ease of use make it an excellent choice for fast prototyping and experimentation.
- PyTorch: PyTorch is an open-source machine learning framework that accelerates the path from research prototyping to production deployment. It provides tensor and deep neural networks computing as high level features. PyTorch is widely used in the field of deep learning due to its dynamic computation graph, which enables fast experimentation and easy debugging of complex models.
- Google Colaboratory: Google Colaboratory is a cloud-based development environment that allows users to write and execute Python code inside the browser, with zero configuration required. It provides access to high-performance GPUs with no charge, making it an ideal choice for training deep neural networks. Colab was utilized for the implementation tasks, enabling the development and training of models on high-performance hardware without requiring the setup of a local development environment.

By utilizing these powerful and flexible tools and technologies, the proposed system was able to achieve high-quality results in the implementation and training of the proposed model.

C. Model Optimization

1) TensorRT Optimization Tools

NVIDIA® TensorRT™ is a software development kit that enables high-performance machine learning inference. It is designed to work in conjunction with training frameworks such as TensorFlow, PyTorch, and MXNet. TensorRT focuses specifically on running a pre-trained neural network efficiently on NVIDIA hardware, delivering approximately 4 to 5 times faster inference than the original model. The main features of this tool include:

- Reduced precision: TensorRT maximizes throughput with FP16 or INT8 by quantizing models while preserving accuracy, allowing for faster computations with less memory usage.
- Layer and tensor fusion: TensorRT optimizes the use of GPU memory and bandwidth by fusing nodes in a kernel, reducing memory access and increasing efficiency.
- Kernel auto-tuning: TensorRT selects the best data layers and algorithms based on the target GPU platform, optimizing performance for a specific hardware configuration.
- Dynamic tensor memory: TensorRT minimizes the memory footprint and reuses memory for tensors efficiently, enabling inference on larger models or multiple models in parallel.
- Multi-stream execution: TensorRT uses a scalable design to process multiple input streams in parallel, improving throughput and reducing latency.
- Time fusion: TensorRT optimizes recurrent neural networks over time steps with dynamically generated ker-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

nels, resulting in faster inference on time-series data.

The various features of TensorRT make it a powerful tool for optimizing and accelerating neural network inference on NVIDIA hardware. By leveraging techniques such as reduced precision, layer and tensor fusion, kernel auto-tuning, dynamic tensor memory, multi-stream execution, and time fusion, it is possible to achieve significant improvements in inference speed without sacrificing accuracy. These optimizations can be particularly useful in scenarios where fast and efficient inference is critical, such as in real-time applications or in systems with limited computational resources. Overall, TensorRT provides a powerful framework for optimizing and accelerating neural network inference on NVIDIA hardware, enabling users to achieve optimal performance and efficiency in their machine learning applications.

2) Optimization Strategy

The process of optimizing and accelerating the neural network inference on NVIDIA hardware involves several steps. Since the implemented model was developed using PyTorch, Torch-TensorRT, the corresponding compiler to target NVIDIA GPUs, was used. First, the obtained PyTorch model was converted to the ONNX format, which is an open format for representing machine learning models. Then, the ONNX model was converted to TensorRT format, taking advantage of the features provided by TensorRT, such as reduced precision, layer and tensor fusion, kernel auto-tuning, dynamic tensor memory, multi-stream execution, and time fusion.

During the conversion process, several optimizations were applied, including int8 and float16 conversions, internal layer fusions, and other techniques that TensorRT features. These optimizations help to maximize throughput while preserving accuracy, resulting in significant improvements in inference speed.

3) Target Runtimes

The target runtimes for this project are Google Colab as a GPU virtual machine for training and tuning the model as a proof of concept, and Jetson Nano with TensorRT support as the deployment edge device. The Google Colab GPU virtual machine is equipped with an Nvidia Tesla T4 GPU with 16GB of memory, an Intel(R) Xeon(R) CPU, and 4 GB of RAM. This configuration accelerates the training process in addition to the advantages that Colaboratory features, such as free access to GPUs. On the other hand, Jetson Nano provides a 128-core NVIDIA Maxwell™ architecture-based GPU, a quad-core ARM® A57 CPU, and 4 GB of RAM, making it a suitable device for deploying the trained model at the edge with limited resources.

V. Results

After the optimization process, the final results on the Jetson Nano edge device with TensorRT support are presented in Table V. The optimized model achieved significant improvements in inference speed without sacrificing accuracy, as demonstrated by the results. Additionally, some examples of the recognized license plates are provided in Figure 19, showcasing the successful deployment of the model on the Jetson Nano device.

Parameter	Ours Before Optim.	Ours After Optim.	Anchor-Free Net [9]
Accuracy (mAP)	90.6	90.2	**97.2**
Speed (FPS)	3.1	**14.2**	8.1

Fig. 19. Examples of the recognized license plates

The proposed ALPR model demonstrated near real-time performance on Jetson Nano with TensorRT support and high accuracy. The end-to-end architecture of the model and its high generalization capability make it suitable for a wide range of ALPR applications. Moreover, the shared feature map and LP region proposals of the proposed model can be exploited to perform other related tasks. In conclusion, the proposed model is a versatile framework that can be adapted and extended to other related computer vision problems.

VI. Conclusion and Future Works

In this paper, we presented the results obtained in our research on Automatic License Plate Recognition (ALPR). We started with a quick overview of ALPR and then conducted a deep study of ALPR state-of-the-art, including ALPR system components and different ALPR techniques. We focused on deep learning-based end-to-end techniques since our contribution comes in this context.

Based on previous efforts, we designed a deep learning-based end-to-end ALPR network that allows vehicle license plates detection and recognition in a single forward pass. This approach has a reduced model size and is more efficient than traditional or deep learning-based two-stage approaches due to sharing convolutional features with both detection and recognition networks and avoiding error accumulation between sequential tasks. The proposed network extracts a shared feature map for both LP detection and recognition modules, an anchor-free detection head that extracts LP centers and associated width and height, and finally, a Region of Interest pooling layer provides the recognition network with LP pooled features that the output is decoded using a beam search algorithm.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

Next, we trained the proposed model on the CCPD dataset using the Adam optimizer for 38 epochs. The model was trained using mini-batches of 32 images and was validated on a separate dataset of 2k images. Due to resource limitations, only 30k images were used for training, but the model proved high accuracy and generalization capability with real-time execution speed on Google Colaboratory platform. The resulting model was optimized using Torch-TensorRT to reduce its size and computational complexity while maintaining high accuracy and generalization capability. We evaluated the model on Jetson Nano edge device with TensorRT support and achieved near real-time execution speed.

To extend this work, we can exploit the designed model for other tasks simultaneously with ALPR, such as vehicle location, brand, color, type, etc., either from the shared feature map shortly as an independent task or by combining it with the obtained LP proposals. Moreover, with the high execution speed of the model, post-processing such as LP tracking and voting processes can be implemented for further accuracy enhancement.

In conclusion, our proposed model achieved high accuracy and real-time execution speed on Jetson Nano edge device with TensorRT support. The deep learning-based end-to-end approach provides a reduced model size and more efficiency than traditional or deep learning-based two-stage approaches, making it a promising framework for ALPR and other related tasks.

Acknowledgment

This work was made possible by NPRP Grant No.: NPRP13S-0206-200273 from the Qatar National Research Fund (a member of The Qatar Foundation). The statements made herein are solely the responsibility of the authors. The publication of this article was funded by Qatar National Library.

References

[1] paperswithcode. Connectionist Temporal Classification Loss . https://paperswithcode.com/method/ctc-loss, 2022. [Online; accessed 25-June-2016].

[2] GeeksForGeeks. Introduction to Beam Search Algorithm. https://www.geeksforgeeks.org/introduction-to-beam-search-algorithm/, 2022. [Online; accessed 25-June-2016].

[3] pyimagesearch. Segmenting characters from license plates. https://customers.pyimagesearch.com/lesson-sample-segmenting-characters-from-license-plates/, 2022. [Online; accessed 25-June-2016].

[4] DataCamp. DataCamp Website. https://www.datacamp.com/tutorial/object-detection-guide, 2022. [Online; accessed 25-June-2016].

[5] Peng Wang Hui Li and Chunhua Shen. *Towards End-to-End Car License Plates Detection and Recognition with Deep Neural Networks*. EEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, Avril 5, 2017.

[6] R. Girshick J. Redmon, S. Divvala and A. Farhadi. *You only look once: Unified, real-time object detectio*. Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 779–788, 2016.

[7] DULANI MEEDENIYA JITHMI SHASHIRANGANA, HESHAN PADMASIRI and CHARITH PERERA2. *Automated License Plate Recognition: A Survey on Methods and Techniques*. IEEE Access (Volume: 9) Page(s): 11203 - 11225, 2021.

[8] Scopus.com. 2000-2021 Results Analysis of (license AND plate AND (detection OR recognition)) keywords. https://www.scopus.com/term/analyzer.uri?sid=65a0db3a5cb7e8af1ed4ea16d3a7fd9b&origin=resultslist&src=s&s=TITLE-ABS-KEY%28license+AND+plate+AND+%28detection+OR+recognition%29%29&sort=plf-f&sdt=b&sot=b&sl=63&count=3599&analyzeResults=Analyze+results&txGid=67431f82b3cd033a59105f60487c4160, 2022. [Online; accessed 13-June-2022].

[9] Sijiang Liu Shuxin Qin. *Towards End-to-end Car License Plate Location and Recognition in Unconstrained Scenarios*. arXiv, Aug 2020.

[10] R. Girshick S. Ren, K. He and J. Sun. *Faster R-CNN: Towards real-time object detection with region proposal networks*. Proc. Adv. Neural Inf. Process. Syst., 2015.

[11] R. Girshick S. Ren, K. He and J. Sun. *Faster r-cnn: Towards real-time object detection with region proposal network*. Advances in neural information processing systems, pp. 91–99, 2015.

[12] D. Erhan C. Szegedy S. Reed C.-Y. Fu W. Liu, D. Anguelov and A. C. Berg. *Ssd: Single shot multibox detector*. European conference on computer vision. Springer, 2016, pp. 21–37, 2016.

[13] Ajin Meng-Nanxue Lu Huan Huang Changchun Ying Zhenbo Xu, Wei Yang and Liusheng Huang. *Towards End-to-End License Plate Detection and Recognition: A Large Dataset and Baseline*. ECCV, 2018.

[14] *Sequence recognition of chinese license plates*. Neurocomputing, vol. 317, pp. 149–158, 2018.

[15] Y. Li B. Li, B. Tian and D. Wen. *Component-based license plate detection using conditional random field model*. IEEE Trans. Intell. Transp. Syst., vol. 14, no. 4, pp. 1690–1699, 2013.

[16] Y. Yao C. Gou, K. Wang and Z. Li. *Vehicle license plate recognition based on extremal regions and restricted boltzmann machines*. IEEE Trans. Intell. Transp. Syst., vol. 17, no. 4, pp. 1096–1107, 2016.

[17] *n iranian license plate recognition system based on color features*. IEEE Trans. Intell. Transp. Syst., vol. 15, no. 4, pp. 1690–1705,, 2014.

[18] G. Faustino G. Alex, F. Santiago and S. JRgen. *Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks*. Proc. Int. Conf. Mach. Learn. (ICML), 2006.

[19] S. Goel and S. Dabas. *Vehicle registration plate recognition system using template matching*. Proc. Int. Conf. Signal Process. Commun. (ICSC), pp. 315–318, 2013.

[20] J. Chen G. Hsu and Y. Chung. *pplication-oriented license plate recognition*. IEEE Trans. Veh. Technol., vol. 62, no. 2, pp. 552–561, 2013.

[21] M. You H. Li, P. Wang and C. Shen. *Reading car license plates using deep neural networks*. Image Vision Comput., vol. 72, pp. 14–23, 2018.

[22] T. Li L. Xun J. Zhang, Y. Li and C. Shan. *icense plate localization in unconstrained scenes using a two-stage cnn-rnn*. IEEE Sensors J., vol. 19, no. 13, pp. 5256–5265,, 2019.

[23] C. Liu and F. Chang. *Hybrid cascade structure for license plate detection in large visual surveillance scenes*. IEEE Trans. Intell. Transp. Syst., vol. 20, no. 6, pp. 2122–2135, 2019.

[24] H. Law and J. Deng. *Cornernet: Detecting objects as paired keypoints*. Proc. Eur. Conf. Comput. Vis. (ECCV), 2018.

[25] L. Jin Y. Liu L. Xie, T. Ahmad and S. Zhang. *new cnn-based method for multi-directional car license plate detection*. EEE Trans. Intell. Transp. Syst., vol. 19, no. 2, pp. 507–517, 2018.

[26] Y. Li M. S. Al-Shemarry and S. Abdulla. *An efficient texture descriptor for the detection of license plates from vehicle images in difficult conditions*. IEEE Trans. Intell. Transp. Syst., vol. 21, no. 2, pp. 553–564, 2020.

[27] L. A. Zanlorensi L. S. Oliveira G. R. Gonalves W. R. Schwartz R. Laroca, E. Severo and D. Menotti. *A robust real-time automatic license plate recognition based on the yolo detector*. Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN), 2018.

[28] S. M. Silva and C. R. Jung. *License plate detection and recognition in unconstrained scenarios*. Proc. Eur. Conf. Comput. Vis. (ECCV), 2018.

[29] U. U. Sheikh S. Setumin and S. A. R. Abu-Bakar. *haracter-based car plate detection and localization*. in Proc. Int. Conf. Information Science, Signal Processing and their Applications, pp. 737–740, 2010.

[30] Q. Zhang C. Liu S. Yu, B. Li and M. Q.-H. Meng. *A novel license plate location method based on wavelet transform and emd analysis*. Pattern Recognit., vol. 48, no. 1, pp. 114 – 125, 2015.

[31] W. Yue and L. Jianmin. *License plate recognition using deep fcn*. Proc. Int. Conf. Cognitive Syst. Signal Process, 2017.

[32] D. Li Y. Yang and Z. Duan. *Chinese vehicle license plate recognition using kernel-based extreme learning machine with deep convolutional features*. IET Intel. Transport Syst., vol. 12, no. 3, pp. 213–219, 2018.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095

[33] Y. Zhao X. Wang X. Hu Y. Yuan, W. Zou and N. Komodakis. *A robust and efficient approach to license plate detection*. IEEE Trans. Image Process., vol. 26, no. 3, pp. 1102–1114, 2017, 2017.
[34] S. Zherzdev and A. Gruzdev. *Lprnet: License plate recognition via deep neural networks*. 2018.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=4436095